

Section 10 — Settings Per Environment

Concept Notes

Applications often need configuration values such as database URLs, API keys, secret tokens, debug settings, and service endpoints.

Hardcoding these values directly inside Python code is difficult to maintain and can expose sensitive information.

Pydantic Settings allows configuration values to be loaded from environment variables and .env files.

BaseSettings

BaseSettings is used to create a structured settings model.

It works similarly to a normal Pydantic model, but its values can come from external configuration sources such as environment variables.

In Pydantic V2, BaseSettings is provided by the separate pydantic-settings package.

Environment Variables

Environment variables store configuration values outside the application source code.

Common examples include:

- Database URLs
- API keys
- Secret keys
- Debug mode
- Server host
- Server port

Pydantic Settings reads these values and validates them according to the declared field types.

.env Files

A .env file stores environment variables in a simple key-value format.

It is useful during local development because configuration can be changed without editing Python source code.

Sensitive .env files should normally be excluded from Git repositories.

Environment-Specific Configuration

Different environments may require different settings.

For example:

- Development
- Testing
- Staging
- Production

The application code can remain the same while configuration values change between environments.

Important Points

- Application configuration should be separated from source code.
- BaseSettings creates validated configuration models.
- In Pydantic V2, install and import from pydantic-settings.
- Settings can be loaded from environment variables.
- Settings can also be loaded from .env files.
- Environment values are validated and converted to declared types.
- Different environments can use different configuration values.
- Secrets should not be hardcoded directly in source code.
- .env files containing secrets should normally be added to .gitignore.
- A .env.example file can document required variables without exposing real secret values.

Syntax

Install Pydantic Settings

`pip install pydantic-settings`

Basic Settings Model

Import BaseSettings

Create Settings Class



Define configuration fields



Create Settings object



Load and validate environment values

Load a .env File

`SettingsConfigDict(`

`env_file = ".env"`

`)`

Basic Flow

Environment Variables / .env



BaseSettings



Validation



Settings Object



Application